نظریه زبان‌ها و ماشین‌ها

زمستان ۱۴۰۴-بهار ۱۴۰۵

جلسه سوم

موضوع:

- مرور جلسه قبل (مفهوم زبان و گرامر)
- چند مثال از گرامرها
- شروع مبحث اتوماتا (فصل ۲):
  - معرفی «پذیرنده‌های متناهی»
  - زبان‌های منظم

مرور جلسات قبل:

مفاهیم الفبا، رشته و زبان را تعریف کردیم.

مفهوم	تعریف	مثال
الفبا	هر مجموعه غیرتهی و متناهی از نمادها	$\sum = \{a, b\}$
رشته	هر دنباله‌ی متناهی از نمادها	$v = abb$
رشته تهی	رشته با طول صفر	$\|\lambda\| = 0$
مجموعه تمام رشته‌ها	تمام رشته‌های یک الفبا	$\sum^* = \{\lambda, a, b, ab, aa, bb, abb, ...\}$
زبان	هر زیر مجموعه‌ای از $\sum^*$	$L = \{a, abb\}$, $L=\{a^n b^n, n \geq 0\}$

گرامر: یک گرامر به صورت یک چهارتایی مرتب تعریف می‌شود

$$G = (V, T, S, P)$$

که در آن:

- V is a finite set of variables (متغیرها)
- T is a finite set of terminal symbols (ترمینال‌ها)
- $S \in V$ is the start variable (متغیر اولیه)
- P is (a finite set of) productions (قوائد تولید)

Productions:

$$x \rightarrow y$$

where

- $x \in (V \cup T)^+$
- $y \in (V \cup T)^*$

یعنی در یک قائده‌ی تولید به شکل بالا:

- x می‌تواند یک رشته‌ی غیرتهی شامل متغیرها و ترمینال‌ها باشد،
- y می‌تواند یک رشته شامل متغیرها و ترمینال‌ها باشد (از جمله رشته‌ی تهی).

Assumptions:

- V and T are non-empty.

- $V \cap T = \{\}$

Derivation (اشتقاق): Given $uxv \in (V \cup T)^+$, the production rule $x \to y$ can be applied to **derive** uyv. We write $uxv \Rightarrow uyv$.
Also, we write $w_1 \Rightarrow^* w_n$ to show that a number of steps (including zero) is taken to derive w_n from w_1.

مثال:

EXAMPLE 1.11

Consider the grammar $G = (\{S\}, \{a, b\}, S, P)$, with P given by

$$S \to aSb,$$
$$S \to \lambda.$$

این گرامر، به عنوان مثال، رشته‌ی $aabb$ را تولید می‌کند. اشتقاق این رشته به این صورت است:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$$

EXAMPLE 1.12

Write a grammar for the language $L = \{a^n b^{n+1} : n \geq 0\}$.

Answer: The grammar $G = (\{S, A\}, \{a, b\}, S, P)$ with productions

$$S \to Ab,$$
$$A \to aAb,$$
$$A \to \lambda.$$

زبانی که یک گرامر تولید می‌کند:

زبان یک گرامر، شامل تمام رشته‌هایی از ترمینال‌هاست که با شروع از متغیر اولیه، مشتق می‌شوند. به عبارت دیگر:

Let $G = (V, T, S, P)$ be a grammar. Then the set

$$L(G) = \{w \in T^* : S \Rightarrow^* w\}$$

is the language generated by G.

تعریف فرم جمله‌ای:

If $w \in L(G)$, then the sequence

$$S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \cdots \Rightarrow w_n \Rightarrow w$$

is a derivation of the string w. The strings $S, w_1, w_2, \ldots, w_n$, which contain variables as well as terminals, are called <u>sentential forms</u>.

EXAMPLE 1.13

A grammar G with productions

$$S \to SS,$$
$$S \to \lambda,$$

$$S \rightarrow aSb,$$

$$S \rightarrow bSa$$

The generated language?

A important example. Read the details, and the proof in the book.

EXAMPLE 1.15
A grammar for **variable identifiers** in C → See in the book

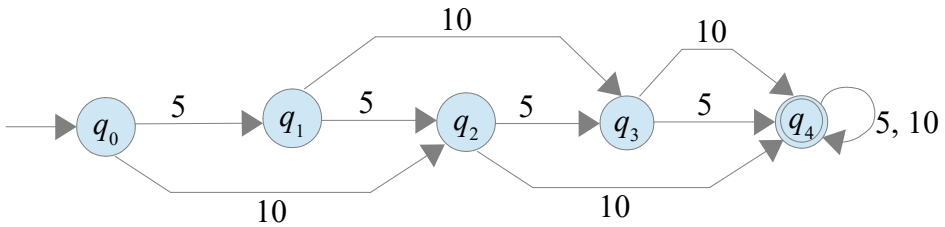
فصل ۲: اتوماتای متناهی (Finite Automata)

در این فصل با مفهوم «اتوماتا»ی متناهی و زبان‌های مرتبط با آن آشنا می‌شویم. برای این منظور، از یک مثال ساده شروع می‌کنیم.

مثال: یک ماشین قهوه سکه‌ای را در نظر بگیرید که سکه‌های ۵ و ۱۰ تومانی می‌گیرد. اگر مجموع سکه‌های انداخته شده در ماشین ۲۰ تومان یا بیشتر باشد، دستگاه یک فنجان قهوه تحویل می‌دهد. می‌توانیم عملکرد کنترلر (کامپیوتر) این دستگاه را با یک ماشین حالت (state machine) توصیف کنیم.

دستگاه همیشه در یکی از چند حالت زیر قرار دارد:

- حالت اولیه: هنوز هیچ سکه‌ای انداخته نشده،
- حالتی که یک سکه‌ی ۵ تومانی انداخته شده،
- حالتی که مجموع سکه‌های انداخته شده ۱۰ تومان است،
- حالتی که مجموع سکه‌های انداخته شده ۱۵ تومان است،
- حالتی که مجموع سکه‌های انداخته شده ۲۰ تومان یا بیشتر است.



در این ماشین حالات:

- حالت اولیه با یک پیکان ورودی به q_0 مشخص شده،
- q_4 «حالت نهایی» یا «حالت پذیرش» است.

بر اساس ماشین حالت طراحی شده، یک دنباله از سکه‌های ورودی، «پذیرفته» می‌شود اگر ماشین حالات را به حالت q_4 ببرد.

به این ماشین حالت، یک «پذیرنده متناهی» (Finite Accepter) می‌گوییم. از آن‌جا که برای هر حالت و هر ورودی، حالت بعدی که ماشین حالت به آن خواهد رفت به‌طور قطعی (deterministic) و یکتا مشخص است، آن را یک پذیرنده‌ی متناهی «قطعی» می‌نامیم.

تعریف پذیرنده‌ی متناهی قطعی:

A **deterministic finite accepter** or **dfa** is defined by the quintuple

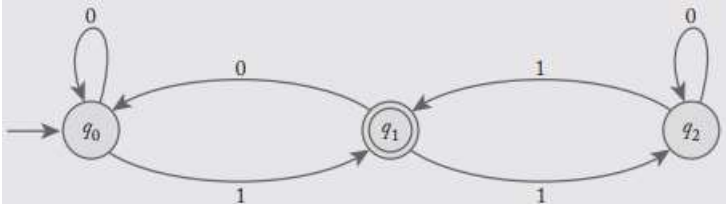
$$M = (Q, \Sigma, \delta, q_0, F),$$

where

- Q is a finite set of states,
- Σ is a finite set of symbols called the input alphabet,
- $\delta : Q \times \Sigma \to Q$ is a <u>total</u> function called the transition function,
- $q_0 \in Q$ is the initial state,
- $F \subseteq Q$ is a set of final states.

EXAMPLE 2.1

گراف زیر را در نظر بگیرید:



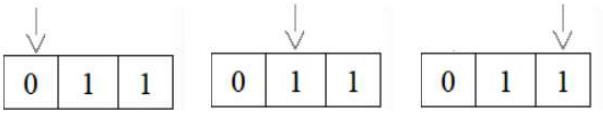
این گراف یک dfa را نمایش می‌دهد:

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\})$$

که در آن تابع گذار به صورت زیر تعریف می‌شود:

$$\delta(q_0, 0) = q_0, \qquad \delta(q_0, 1) = q_1,$$
$$\delta(q_1, 0) = q_0, \qquad \delta(q_1, 1) = q_2,$$
$$\delta(q_2, 0) = q_2, \qquad \delta(q_2, 1) = q_1$$

نحوه‌ی اجرای dfa روی یک رشته‌ی ورودی: v=011



تابع گذار را در قالب یک جدول هم می‌توان نمایش داد.

	0	1
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_2	q_1

این پذیرنده، رشته‌ی 01 را می‌پذیرد ولی رشته‌ی 100 یا 1100 را نمی‌پذیرد.

زبان یک dfa: مجموعه‌ی تمام رشته‌هایی که یک dfa می‌پذیرد، «زبان» آن dfa را تشکیل می‌دهد. تعریف دقیق‌تر زبان یک dfa را با استفاده از «تعمیم» تابع گذار (extended transition function) انجام می‌دهیم.

Extended transition function: $\delta^* : Q \times \Sigma^* \to Q$

Assume a transition function containing:

$$\delta(q_0, a) = q_1$$
$$\delta(q_1, b) = q_2,$$

We can summarize it as $\delta^*(q_0, ab) = q_2$

So, δ^* can be defined recursively:

$$\delta^*(q, \lambda) = q, \qquad |\lambda| = 0$$
$$\delta^*(q, a) = \delta(q, a), \qquad |a| = 1$$
$$\delta^*(q, wa) = \delta(\delta^*(q, w), a) \qquad |wa| \geq 2$$

For example, $\delta^*(q_0, ab) = \delta(\delta^*(q_0, a), b) = \delta(q_1, b) = q_2$

A string w is **accepted** by a dfa $M = (Q, \Sigma, \delta, q_0, F)$ if $\delta^*(q_0, w) \in F$.

The **language** accepted by a dfa $M = (Q, \Sigma, \delta, q_0, F)$ is the set of all strings on Σ accepted by M. In formal notation,

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$

به بیان ساده،

$$\delta^*(q_0, w)$$

تابعی است که نشان می‌دهد dfa با شروع از حالت اولیه، پس از دیدن رشته‌ی w، در نهایت در چه حالتی خواهد بود.

طبق تعریف، δ و δ^* تابع‌های تام (total) هستند. به ازای هر نماد ورودی، دقیقاً یک حرکت تعریف شده و به ازای هر رشته، و با شروع از حالت اولیه، در انتها دقیقاً به یک حالت خواهیم رسید. از این رو، این پذیرنده‌ها را «قطعی» (deterministic) می‌گوییم.

سوال: برای یک پذیرنده‌ی متناهی قطعی M، مکل زبان $L(M)$ شامل چه رشته‌هایی می‌شود؟

EXAMPLE 2.2

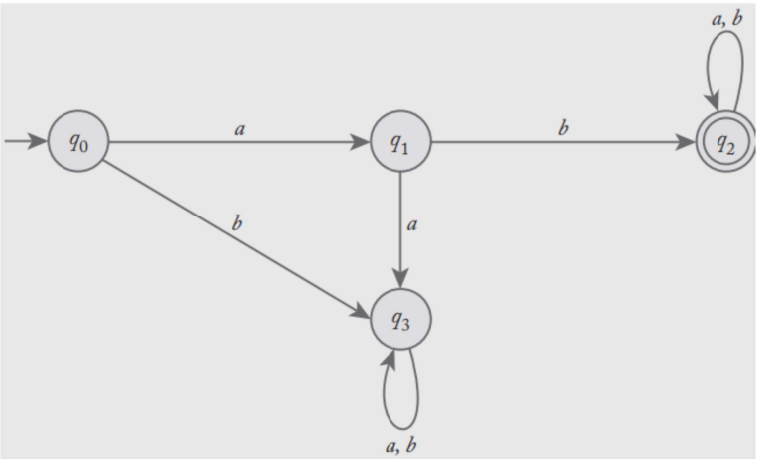
زبانی که این dfa می‌پذیرد؟

$$L = \{a^n b : n \geq 0\}$$

در این پذیرنده‌ی متناهی، q_2 یک حالت trap است.

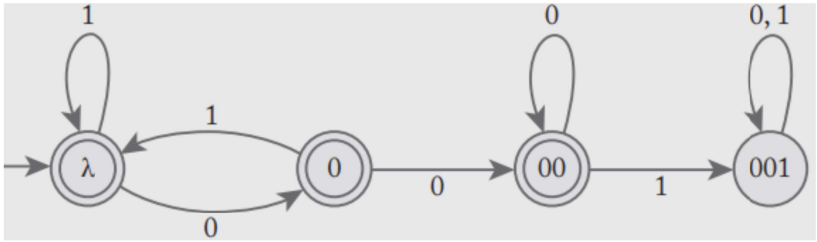
EXAMPLE 2.3

A deterministic finite accepter (dfa) that recognizes the set of all strings on $\Sigma = \{a, b\}$ starting with the prefix ab.



استفاده از گراف جهت‌دار برای نمایش dfa: در مثال‌های بالا، از گراف برای نمایش dfa استفاده کردیم. باید دقت کرد که dfa یک گراف نیست ولی برای هر dfa می‌توان یک گراف معادل در نظر گرفت که حالت‌ها و گذارهای dfa را نمایش می‌دهد و برای آسانتر کردن فهم مفاهیم و استدلال‌ها به کار می‌رود.

EXAMPLE 2.4
Find a dfa that accepts all the strings on {0, 1}, except those containing the substring 001.



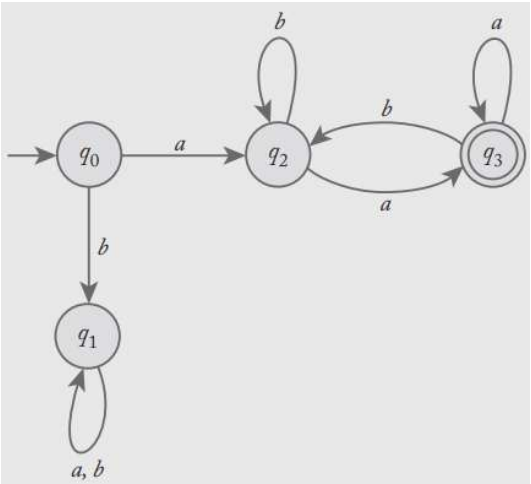
«زبان‌های منظم»

سوال: آیا برای هر زبان دلخواه، می‌توان یک dfa طراحی کرد؟ (= آیا زبانی وجود دارد که نتوان هیچ dfa ای برای آن ساخت؟)

مجموعه‌ی تمام dfa ها را در نظر بگیرید. مجموعه‌ی تمام زبان‌هایی که این dfaها می‌پذیرند، خانواده‌ی «زبان‌های منظم» را تشکیل می‌دهند.

DEFINITION 2.3
A language L is called **regular** if and only if there exists some deterministic finite accepter M such that $L = L(M)$

مثال (EXAMPLE 2.5): نشان دهید زبان زیر، منظم است:

$$L = \{awa : w \in \{a, b\}^*\}$$

برای این کار، کافی است یک dfa برای این زبان پیدا کنیم.